# Tutoriumsblatt 10

## Agent-Loop, ReAct und Planung

Bearbeitungszeit etwa 90 Minuten · 40 Punkte · Lösungen im Anhang ab Seite 6

**Nach diesem Blatt kannst du**

- begründen, ob eine Aufgabe einen Workflow oder einen Agenten braucht,
- den Agent-Loop als Zustandsfolge verfolgen und Notausgänge prüfen,
- Input-Tokens über viele Runden herleiten und Planer-Tokens einrechnen,
- Erfolgswahrscheinlichkeit und Rundenzahl mehrschrittiger Läufe abschätzen,
- einen Trace prüfen, ReAct-Gedanken bewerten und messen, ob Planung hilft.

**So arbeitest du mit dem Blatt**

Bearbeite die Aufgaben der Reihe nach, sie werden schwieriger. Taschenrechner genügt, rechne mit drei Nachkommastellen. Alle Zahlen, Traces und Messergebnisse sind ausgedachte Beispiele. Kosten wie `kosten_cent` im Notebook: 1 bzw. 5 US-Dollar pro Million Input- bzw. Output-Tokens. Notebook: Aufgabe 1 gehört zu TODO 1, Aufgabe 2 zu TODO 2 und 3, Aufgabe 3 zu B3 und TODO 7, Aufgabe 5 zu TODO 4, 5 und 8.

### Aufgabe 1 Workflow oder Agent?

5 Punkte · etwa 10 Min.

Bezug: Modul 10, Lerneinheit 1 (Workflow oder Agent?)

Die Nordwerk GmbH sammelt Ideen für ihren Instandhaltungs-Assistenten.

(a) Workflow oder Agent? Begründe mit der Entscheidungsfrage, bei Workflows mit Muster. [4 P]

(i) Nach jeder Schicht die neuen Tickets in fünf Sätzen für die Übergabe zusammenfassen.

(ii) Mails von Servicepartnern als Angebot, Termin oder Rechnung einordnen und weiterleiten.

(iii) „Halle 2 verbraucht seit Dienstag mehr Druckluft. Finde heraus, woran das liegt."

(iv) Jeden Montag je Maschine Historie und Handbuch prüfen: Ist eine Wartung fällig?

(b) Ein Kollege sagt: „Der Montagsbericht aus (iv) macht 16 Tool-Aufrufe. Das ist doch ein Agent." Widerlege die Aussage und sage, warum du im Zweifel mit dem Workflow beginnst. [1 P]

### Aufgabe 2 Der Loop als Zustandsfolge

7 Punkte · etwa 15 Min.

Bezug: Modul 10, Lerneinheiten 2 (Der Agent-Loop Schritt für Schritt) und 3 (Abbruchbedingungen)

Unten steht der Loop, gegenüber dem Notebook vereinfacht (ohne Trace und Tokens, Limit 5 statt 10). Wie in TODO 2 zählt `wiederholt_sich`, wie oft derselbe Aufruf (Name und Eingabe, Schlüssel sortiert) schon in `aufrufe` steht, und meldet ab `max_gleich = 2` Treffern eine Schleife.

Den Zustand nach Runde $k$ beschreibst du durch $z_k = (m_k,\ a_k,\ \text{Status})$ mit $m_k$ Nachrichten im Verlauf und $a_k$ ausgeführten Tool-Aufrufen. Start: $z_0 = (2,\ 0,\ \text{läuft})$.

```python
def agent(aufgabe, system, max_schritte=5):
    nachrichten = [system_nachricht(system), user_nachricht(aufgabe)]
    aufrufe = []                                  # ausgeführte Aufrufe
    for schritt in range(1, max_schritte + 1):
        msg = modell(nachrichten)                 # ein API-Aufruf
        nachrichten.append(msg)
        if not msg.tool_calls:
            return "fertig", schritt
        for tc in msg.tool_calls:
            if wiederholt_sich(aufrufe, tc.name, tc.eingabe):
                return "schleife", schritt
            ergebnis = fuehre_tool_aus(tc.name, tc.eingabe)
            aufrufe.append((tc.name, tc.eingabe))
            nachrichten.append(tool_nachricht(tc.id, ergebnis))
    return "limit", max_schritte
```

(a) Diagnose E21 an M-05: Runde 1 ruft zwei Tools parallel auf (Status, Sensordaten), Runde 2 die Handbuch-Suche, Runde 3 parallel die Historie und eine zweite Handbuch-Suche. In Runde 4 antwortet das Modell ohne Tool-Aufruf. Gib $z_1$ bis $z_4$ an. Wie viele Nachrichten schickt der Loop insgesamt an das Modell? [2 P]

(b) Jemand rückt die Zeile, die die `tool`-Nachricht anhängt, versehentlich eine Ebene nach links, hinter die innere `for`-Schleife. Was passiert im Lauf aus (a), in welcher Runde? [1 P]

(c) *Schreibtischtest:* Ein Agent soll M-01 prüfen. Das Modell liefert in jeder Runde genau einen dieser Aufrufe:

```
R1: get_sensor_data          {"machine_id": "M-01"}
R2: get_maintenance_history  {"machine_id": "M-01", "limit": 3}
R3: get_sensor_data          {"machine_id": "M-01"}
R4: get_maintenance_history  {"limit": 3, "machine_id": "M-01"}
R5: get_maintenance_history  {"machine_id": "M-01", "limit": 3}
R6: get_sensor_data          {"machine_id": "M-01"}
```

(i) Gib für jede Runde an, wie viele gleiche Aufrufe `wiederholt_sich` zählt, und mit welchem Status der Loop nach welcher Runde endet. (ii) Wie endet der Lauf, wenn `wiederholt_sich` die Eingaben mit `str(eingabe)` statt mit sortierten Schlüsseln vergleicht? [3 P]

(d) Warum gibt der Loop bei `schleife` und `limit` einen Status zurück, statt eine Ausnahme zu werfen? Was sollte das umgebende System damit tun? [1 P]

## Aufgabe 3 Kosten über viele Runden, Budget und Planer

9 Punkte · etwa 20 Min.

Bezug: Modul 10, Lerneinheiten 3 (Abbruchbedingungen und Kosten) und 6 (Planung mit strukturiertem Plan)

Ein Agent hat einen festen Anfang von $T_0 = 3000$ Tokens (System-Prompt, Tool-Definitionen, Aufgabe). Jede Runde hängt im Mittel $\Delta = 1000$ Tokens an den Verlauf an. Nach der Lerneinheit verbraucht er über $n$ Runden

$$S(n) = \sum_{k=1}^{n} \left(T_0 + (k-1)\,\Delta\right) = n\,T_0 + \frac{n(n-1)}{2}\,\Delta \quad \text{Input-Tokens.}$$

(a) Berechne $S(5)$ und $S(10)$. [1 P]

(b) Leite allgemein her: $\dfrac{S(2n)}{S(n)} = \dfrac{2\,(2T_0 + (2n-1)\,\Delta)}{2T_0 + (n-1)\,\Delta}$. Gegen welchen Wert strebt das Verhältnis für $n \to \infty$, welchen Wert hat es für $\Delta = 0$? Deute beide Grenzfälle in je einem Satz. [2 P]

(c) *Budget wie in Bonus B3:* Nach jeder Runde bricht der Loop mit Status `budget` ab, wenn die bisherigen Input-Tokens 30 000 überschreiten. Nach welcher Runde greift das, und wie viele Tokens sind dann verbraucht? Bestimme das größte $n$ mit $S(n) \le 30\,000$ auch über die quadratische Ungleichung. [2 P]

(d) *Planer-Tokens:* Der Planer-Aufruf kostet 1500 Input-Tokens (Tool-Liste, Aufgabe) und 200 Output-Tokens (der Plan). Der Plan steht in der ersten Nachricht und geht in jeder Runde mit, $T_0$ steigt also um 200. Jede Runde des Loops erzeugt 150 Output-Tokens. Berechne die Kosten $(T_{\text{in}} + 5\,T_{\text{out}})/10^6 \cdot 100$ in Cent für je 6 Runden (i) ohne Plan, (ii) mit Plan samt Planer-Aufruf und (iii) mit Plan ohne Planer-Aufruf, so wie das Notebook misst. Welche Mehrkosten in Prozent zeigt die Messung, welche entstehen wirklich? [3 P]

(e) Angenommen, der Plan spart eine Runde: 5 Runden mit Plan gegen 6 Runden ohne. Ist die Variante mit Plan dann insgesamt billiger? [1 P]

## Aufgabe 4 Wie zuverlässig ist ein Lauf mit vielen Schritten? 9 Punkte · etwa 20 Min.

Bezug: Modul 10, Lerneinheiten 3 (Abbruchbedingungen) und 4 (typische Fehler); dazu Pfadregeln aus der Schule

Ein vereinfachtes Modell mit ausgedachten Wahrscheinlichkeiten, echte Werte liefert nur das Eval: Ein Diagnoselauf für M-07 braucht vier Tool-Schritte nacheinander (Status, Sensordaten, Historie, Handbuch) und danach eine Antwortrunde. Pro Runde gibt es genau einen Tool-Aufruf. Jeder Versuch eines Schritts endet unabhängig von allen anderen *richtig* (Wahrscheinlichkeit $p = 0{,}94$), mit einem *sichtbaren Fehler* ($e = 0{,}05$: erfundenes Tool oder falscher Parameter, `fuehre_tool_aus` meldet `FEHLER`) oder mit einem *stillen Fehler* ($u = 0{,}01$: etwa die falsche Maschine abgefragt; niemand merkt es, der Agent macht normal weiter, die Antwort wird aber falsch).

(a) Angenommen, sichtbare Fehler ließen sich nicht korrigieren, jeder `FEHLER` macht den Lauf wertlos. Berechne die Erfolgswahrscheinlichkeit für vier und für acht Schritte (etwa ein Schritt pro Maschine). Warum entfällt dieses Risiko für Tool-Aufrufe im Workflow (TODO 1)? [2 P]

(b) Jetzt meldet `fuehre_tool_aus` sichtbare Fehler mit hilfreicher Meldung zurück, und der Agent versucht den Schritt höchstens $r$-mal erneut (danach ist der Lauf wertlos). Leite mit $\sum_{i=0}^{r} e^i = (1 - e^{\,r+1})/(1-e)$ her, dass ein Schritt dann mit $P_r = p\,(1 - e^{\,r+1})/(1-e)$ gelingt. Berechne $P_1$, den Grenzwert $P_\infty$ für $r \to \infty$ und jeweils die Erfolgswahrscheinlichkeit eines Laufs mit vier Schritten. Welcher Anteil der Läufe scheitert auch bei beliebig vielen Wiederholungen, und warum? [3 P]

(c) Ohne Begrenzung der Wiederholungen hat die Zahl der Versuche pro Schritt den Erwartungswert $1/(1-e)$ (geometrische Verteilung). Berechne die erwartete Rundenzahl eines Laufs samt Antwortrunde. Warum ändern stille Fehler die Rundenzahl nicht, und was folgt daraus für den Status `fertig`? [2 P]

(d) Das Iterationslimit sei `max_schritte = 6`. Wie viele sichtbare Fehler darf ein Lauf höchstens haben, damit er noch antwortet? Berechne die Wahrscheinlichkeit, dass ein Lauf mit Status `limit` endet. Zur Kontrolle: Genau ein sichtbarer Fehler im ganzen Lauf hat die Wahrscheinlichkeit $4e(1-e)^4$. [2 P]

## Aufgabe 5 Trace prüfen, ReAct und Planung messen

10 Punkte · etwa 22 Min.

Bezug: Modul 10, Lerneinheiten 4 (Einen Trace lesen), 5 (ReAct) und 7 (Messen, ob Planung hilft)

Notebook-Aufgabe A2: „Die Drehmaschine vibriert stärker als sonst. Finde die wahrscheinliche Ursache und sag, was zu tun ist.“ Ein möglicher Trace mit ReAct-Prompt (beispielhaft, gekürzt):

| Runde | Art | Inhalt |
|---|---|---|
| 1 | Gedanke | Ich prüfe zuerst die Messwerte der Drehmaschine. |
| 1 | Aktion | `get_sensor_data(M-03)` |
| 1 | Beobachtung | Vibration Spindel 4,8 mm/s (max. 4,5) ÜBERSCHRITTEN; Spindeltemperatur 63 °C (max. 70) ok; Hydraulikdruck 41 bar (min. 38) ok |
| 2 | Gedanke | Die Vibration ist zu hoch. Ich sehe mir die Messwerte noch einmal genauer an. |
| 2 | Aktion | `get_sensor_data(M-03)` |
| 2 | Beobachtung | dieselben Werte wie in Runde 1 |
| 3 | Gedanke | Wann wurden die Spindellager zuletzt gewartet? |
| 3 | Aktion | `get_maintenance_history(M-03)` |
| 3 | Beobachtung | 28.07.2026 Spindellager nachgeschmiert (NW-LX2); 28.07.2026 Spindelrundlauf 7 µm, nahe am Grenzwert 8 µm; 30.06.2026 Spannfutter gereinigt |
| 4 | Antwort | Befund: Vibration 4,8 mm/s über dem Grenzwert 4,5, Spindeltemperatur mit 63 °C kritisch hoch. Ursache: Spindellager verschlissen, Rundlauf zuletzt 7 µm. Maßnahme: laut Handbuch sofort stillsetzen und Spindellagersatz HTD-1100 einbauen, Ticket mit Priorität hoch anlegen. |

(a) Prüfe den Trace mit den drei Fragen der Lerneinheit, die dritte als Tabelle (Aussage, Beleg im Trace, gedeckt?). Welche zwei Tools fehlen, und wozu? [4 P]

(b) Hätte die Schleifenerkennung mit `max_gleich = 2` den Aufruf in Runde 2 verhindert? Begründe mit der Zählung. [1 P]

(c) Ein Kollege: „Der Gedanke in Runde 2 erklärt den zweiten Aufruf, also war er sinnvoll.“ Nimm Stellung und formuliere einen besseren Gedanken für Runde 2 nach dem ReAct-Muster. [2 P]

(d) *Messen:* ReAct gegen ReAct mit Plan auf A1 bis A6 aus dem Notebook (A4: alle acht Maschinen, A6: Negativfall „Montagelinie B“) und A7, A8 aus dem Projekt (A7: alle Maschinen in Halle 2):

| Variante | A1 | A2 | A3 | A4 | A5 | A6 | A7 | A8 | Erfolg | Ø Tokens im Loop |
|---|---|---|---|---|---|---|---|---|---|---|
| ReAct | ✓ | ✓ | ✓ | × | ✓ | × | × | ✓ | 5/8 | 8000 |
| ReAct + Plan | ✓ | ✓ | ✓ | ✓ | ✓ | × | ✓ | ✓ | 7/8 | 10 000 |

Wilson-Intervalle (95 %): 31 bis 86 % bzw. 53 bis 98 %. Der Planer kostet pro Aufgabe etwa 1700 Tokens zusätzlich (nicht in der Tabelle). Bestimme $b$ (nur ReAct erfolgreich), $c$ (nur mit Plan erfolgreich) und den p-Wert des exakten McNemar-Tests (Modul 9). Berechne die Mehrkosten ohne und mit Planer. Entscheide begründet und nenne den nächsten Schritt. [3 P]

## Aufgabe 6 (⋆) Geometrische Verteilung und Iterationslimit

Zusatz · etwa 10 Min.

*Vertiefung über das Modul hinaus.* Ein Versuch gelingt mit Wahrscheinlichkeit $q$, sonst wird wiederholt. $N$ zählt die Versuche bis zum ersten Erfolg: $P(N = k) = (1 - q)^{k-1}\, q$.

(a) Leite $E(N) = 1/q$ her. Hinweis: Leite $\sum_{k \geq 0} x^k = 1/(1 - x)$ für $|x| < 1$ nach $x$ ab.

(b) $F$ zählt die sichtbaren Fehler eines Laufs aus Aufgabe 4. Begründe $P(F = f) = \binom{f+3}{3} e^f (1-e)^4$ und berechne $P(\texttt{limit})$ für die Limits 7 und 8. Welches wählst du?

# Anhang: Lösungen

Erst selbst rechnen, dann vergleichen. Die Lösungen zeigen den Rechenweg und typische Fehler.

## Lösung zu Aufgabe 1 Workflow oder Agent?

(a) Entscheidungsfrage: Kennst du die Schritte vorab? Dann reicht ein Workflow. Hängen sie vom Zwischenergebnis ab, brauchst du einen Agenten.

(i) **Workflow.** Die Schritte sind immer gleich: Der Code ruft die Tickets ab, ein einzelner Modellaufruf fasst zusammen, wie in TODO 1. Streng genommen braucht das keines der fünf Muster. Prompt Chaining gibt ebenfalls volle Punkte, etwa erst jedes Ticket verdichten, dann die Übergabe schreiben.

(ii) **Workflow, Routing.** Eine Eingabe wird klassifiziert und auf einen festen Pfad geschickt. Welche Pfade es gibt, steht vorab fest.

(iii) **Agent.** Welche Daten nötig sind, hängt von jedem Befund ab: zuerst etwa die Druckluftwerte der Maschinen, je nach Ergebnis dann Tickets, Historie oder das Handbuch zur Leckagesuche. Die Schritte lassen sich nicht vorab festlegen.

(iv) **Workflow, Parallelisierung.** Für jede Maschine laufen dieselben Schritte (Historie, Intervall im Handbuch), die Ergebnisse werden zu einer Liste zusammengeführt.

(b) Entscheidend ist nicht die Zahl der Tool-Aufrufe, sondern wer die Reihenfolge bestimmt. Beim Montagsbericht legt der Code alle 16 Aufrufe fest, also ist es ein Workflow. Workflows sind vorhersagbarer, billiger und schneller. Deshalb gilt die Empfehlung aus „Building Effective AI Agents“: mit der einfachsten Lösung beginnen und Komplexität nur hinzufügen, wenn sie nachweislich hilft.

> **Typischer Fehler**
>
> Viele Tool-Aufrufe für einen Agenten halten. Auch ein Workflow kann Dutzende Tools aufrufen. Ein Agent liegt erst vor, wenn das Modell selbst entscheidet, was als Nächstes kommt.

**Bewertung:** (a) je Aufgabe 1 P für Entscheidung mit Begründung, bei Workflows mit Muster. (b) 1 P.

## Lösung zu Aufgabe 2 Der Loop als Zustandsfolge

(a) Jede Runde hängt eine Assistenten-Nachricht und je Tool-Aufruf eine `tool`-Nachricht an. Der API-Aufruf in Runde $k$ schickt den Verlauf $m_{k-1}$.

| Runde | Modell liefert | Zustand danach | gesendet |
|---|---|---|---|
| 1 | 2 Tool-Aufrufe | $z_1 = (5,\ 2,\ \text{läuft})$ | 2 |
| 2 | 1 Tool-Aufruf | $z_2 = (7,\ 3,\ \text{läuft})$ | 5 |
| 3 | 2 Tool-Aufrufe | $z_3 = (10,\ 5,\ \text{läuft})$ | 7 |
| 4 | Antwort | $z_4 = (11,\ 5,\ \text{fertig})$ | 10 |

Zusammen schickt der Loop $2 + 5 + 7 + 10 = \boxed{24}$ Nachrichten an das Modell, weil die API zustandslos ist und jede Runde den ganzen Verlauf erneut braucht.

(b) Nach der inneren Schleife zeigen `tc` und `ergebnis` nur noch auf den letzten Aufruf. In Runde 1 bekommt deshalb nur der Sensordaten-Aufruf eine `tool`-Nachricht, die Antwort zum Status-Aufruf fehlt. Beim API-Aufruf in **Runde 2** lehnt die API den Verlauf mit einem 400-Fehler zur `tool_call_id` ab. Regel: Jeder Eintrag in `tool_calls` braucht genau eine `tool`-Nachricht mit seiner ID.

(c) (i) Mit sortierten Schlüsseln zählt `wiederholt_sich` vor dem jeweiligen Aufruf:

| Runde | Aufruf | gleiche vorher | Folge |
|---|---|---|---|
| R1 | Sensordaten | 0 | ausgeführt |
| R2 | Historie | 0 | ausgeführt |
| R3 | Sensordaten | 1 (R1) | ausgeführt |
| R4 | Historie, Schlüssel vertauscht | 1 (R2) | ausgeführt |
| R5 | Historie | 2 (R2, R4) | Abbruch |

Der Loop endet in **Runde 5 mit Status `schleife`**, das Tool wird nicht ein drittes Mal ausgeführt. R4 zählt als gleich, weil die Reihenfolge der Schlüssel nach dem Sortieren keine Rolle spielt.

(ii) Mit `str(eingabe)` sehen R2 und R4 verschieden aus. R4 zählt 0, R5 zählt nur 1 (R2) und wird ausgeführt. Danach ist `max_schritte = 5` erreicht, der Lauf endet mit **Status `limit` nach Runde 5**. R6 hätte die Schleife zwar erkannt, kommt aber nicht mehr dran. Der Fehler versteckt also eine Schleife, kostet eine zusätzliche Runde und liefert einen irreführenden Status.

(d) Ein Abbruch ist kein Absturz. Mit einem Status bleiben Trace und Verlauf erhalten, und das umgebende System entscheidet, wie es weitergeht: etwa einen Menschen informieren, den Lauf protokollieren und im Eval als Abbruch getrennt zählen. Eine Ausnahme würde den Lauf ohne Ergebnis beenden und müsste überall abgefangen werden.

> **Typischer Fehler**
>
> Den aktuellen Aufruf mitzählen und schon in R3 eine Schleife melden. `max_gleich = 2` heißt: Erst der dritte identische Aufruf löst den Abbruch aus, weil dann zwei gleiche schon in der Liste stehen.

**Bewertung:** (a) $z_1$ bis $z_4$ 1 P, Summe 24 1 P. (b) 1 P. (c) (i) Zählung 1 P, Status und Runde 1 P, (ii) 1 P. (d) 1 P.

## Lösung zu Aufgabe 3 Kosten über viele Runden, Budget und Planer

(a) $S(5) = 5 \cdot 3000 + \frac{5 \cdot 4}{2} \cdot 1000 = 15\,000 + 10\,000 = \boxed{25\,000}$ und $S(10) = 30\,000 + 45 \cdot 1000 = \boxed{75\,000}$.

(b) Es ist $S(2n) = 2n\,T_0 + \frac{2n(2n-1)}{2}\,\Delta = 2n\,T_0 + n(2n-1)\,\Delta$. Kürzen durch $n$ und Erweitern mit 2 ergibt

$$\frac{S(2n)}{S(n)} = \frac{2T_0 + (2n-1)\,\Delta}{T_0 + \frac{n-1}{2}\,\Delta} = \frac{2\,(2T_0 + (2n-1)\,\Delta)}{2T_0 + (n-1)\,\Delta}.$$

Für $n \to \infty$ teilst du Zähler und Nenner durch $n$: Es bleibt $2 \cdot 2\Delta/\Delta = \boxed{4}$. Für $\Delta = 0$ ist das Verhältnis $2 \cdot 2T_0/(2T_0) = \boxed{2}$. Probe mit $n = 5$: $2\,(6000 + 9000)/(6000 + 4000) = 3 = 75\,000/25\,000$.

Deutung: Bei langen Läufen dominiert der wachsende Verlauf, doppelt so viele Runden kosten fast das Vierfache (quadratisches Wachstum). Ohne wachsenden Verlauf wären die Kosten linear, doppelte Rundenzahl hieße doppelte Kosten.

(c) $S(5) = 25\,000 \le 30\,000$, aber $S(6) = 18\,000 + 15\,000 = 33\,000 > 30\,000$. Das Budget greift **nach Runde 6**, verbraucht sind dann $\boxed{33\,000}$ Input-Tokens. Weil erst nach der Runde geprüft wird, überschreitet der Agent das Budget um bis zu eine Runde.

Quadratische Ungleichung:

$$3000\,n + 500\,n(n-1) \le 30\,000 \iff 500\,n^2 + 2500\,n - 30\,000 \le 0 \iff n^2 + 5n - 60 \le 0.$$

Die positive Nullstelle ist $n = \frac{-5+\sqrt{25+240}}{2} = \frac{-5+16{,}279}{2} \approx 5{,}64$. Die größte ganze Rundenzahl im Budget ist also $\boxed{n = 5}$.

(d) Der Plan bringt in jeder Runde 200 Tokens mehr, über 6 Runden $6 \cdot 200 = 1200$ (das ist der Aufschlag $n \cdot T_{\text{Plan}}$ aus der Lerneinheit).

| Variante (6 Runden) | $T_{\text{in}}$ | $T_{\text{out}}$ | Cent $= (T_{\text{in}} + 5\,T_{\text{out}})/10^4$ |
|---|---|---|---|
| (i) ohne Plan | 33 000 | 900 | $(33\,000 + 4500)/10^4 = 3{,}75$ |
| (ii) mit Plan und Planer | 35 700 | 1100 | $(35\,700 + 5500)/10^4 = 4{,}12$ |
| (iii) mit Plan, ohne Planer | 34 200 | 900 | $(34\,200 + 4500)/10^4 = 3{,}87$ |

Dabei ist $34\,200 = 6 \cdot 3200 + 15 \cdot 1000$ der Loop mit Plan, dazu kommen beim Planer 1500 Input- und 200 Output-Tokens. Variante (iii) entspricht der Messung im Notebook. Die Messung zeigt $3{,}87/3{,}75 - 1 = \boxed{3{,}2\,\%}$ Mehrkosten, wirklich sind es $4{,}12/3{,}75 - 1 = \boxed{9{,}9\,\%}$. Ohne den Planer-Aufruf werden die Mehrkosten also etwa um den Faktor 3 unterschätzt.

(e) 5 Runden mit Plan: Loop $5 \cdot 3200 + 10 \cdot 1000 = 26\,000$, mit Planer 27 500 Input-Tokens; Output $5 \cdot 150 + 200 = 950$. Kosten $(27\,500 + 4750)/10^4 = \boxed{3{,}225 \text{ Cent}} < 3{,}75$ Cent. **Ja**, die Variante mit Plan ist dann um 14 % billiger. Die eingesparte Runde ist die teuerste, allein ihr Input sind $T_0 + 5\Delta = 8000$ Tokens.

> **Typischer Fehler**
>
> Den Plan nur einmal zählen. Er steht in der ersten Nachricht und wird deshalb in jeder Runde erneut mitgeschickt. Ebenso häufig: den Planer-Aufruf vergessen, wie es die Messung im Notebook tut.

**Bewertung:** (a) 1 P. (b) Umformung 1 P, beide Grenzwerte mit Deutung 1 P. (c) Runde 6 und 33 000 Tokens 1 P, quadratische Ungleichung 1 P. (d) (i) 1 P, (ii) 1 P, (iii) mit beiden Prozentwerten 1 P. (e) 1 P. Folgefehler werden nicht doppelt abgezogen.

## Lösung zu Aufgabe 4 Wie zuverlässig ist ein Lauf mit vielen Schritten?

(a) Die Schritte sind unabhängig, also multiplizieren sich die Wahrscheinlichkeiten: $0{,}94^4 = \boxed{0{,}781}$ und $0{,}94^8 = \boxed{0{,}610}$. Bei acht Schritten scheitern schon fast 40 % der Läufe.

Im Workflow ruft der Code die Tools mit festen Namen und Parametern auf. Erfundene Tools oder falsche Parameter kommen dort nicht vor, und kein Schritt kann vergessen werden. Fehleranfällig bleibt nur der Modellaufruf am Ende. Jede Entscheidung, die das Modell selbst trifft, ist eine weitere Fehlerquelle. Das ist ein Grund für die Empfehlung, mit der einfachsten Lösung zu beginnen.

(b) Der Schritt gelingt beim $(i+1)$-ten Versuch genau dann, wenn die ersten $i$ Versuche sichtbare Fehler waren und der $(i+1)$-te richtig ist. Wegen der Unabhängigkeit hat das die Wahrscheinlichkeit $e^i p$. Die Fälle $i = 0, \ldots, r$ schließen sich gegenseitig aus, also gilt mit der endlichen geometrischen Reihe

$$P_r = \sum_{i=0}^{r} e^i \, p = p \cdot \frac{1 - e^{r+1}}{1 - e}.$$

Damit ist $P_1 = 0{,}94 \cdot (1 + 0{,}05) = \boxed{0{,}987}$ und wegen $e^{r+1} \to 0$

$$P_\infty = \frac{p}{1 - e} = \frac{0{,}94}{0{,}95} = \boxed{0{,}9895}.$$

Lauf mit vier Schritten: $0{,}987^4 = 0{,}949$ bei einer Wiederholung, $0{,}9895^4 = 0{,}959$ bei beliebig vielen. Auch dann scheitern $1 - 0{,}959 \approx \boxed{4{,}1\,\%}$ der Läufe, und zwar an *stillen* Fehlern: Niemand bemerkt sie, also wird nichts wiederholt. Es gilt $P_\infty = p/(p+u)$, nur der Anteil der stillen Fehler bleibt übrig. Solche Fehler fängt nur die Prüfung von Antwort und Trace ab, etwa die Frage, ob jede Aussage durch eine Beobachtung gedeckt ist, und das Eval gegen eine Referenz.

(c) Erwartete Versuche pro Schritt: $1/0{,}95 = 1{,}053$. Mit der Linearität des Erwartungswerts sind es $4 \cdot 1{,}053 + 1 = \boxed{5{,}21}$ Runden im Mittel.

Ein stiller Fehler beendet einen Versuch wie ein richtiges Ergebnis: Es gibt kein `FEHLER`, keine Wiederholung, der Loop geht normal weiter. Ein Lauf mit stillem Fehler braucht also genauso viele Runden und endet ebenfalls mit Status `fertig`. Daraus folgt: `fertig` heißt nur, dass das Modell ohne Tool-Aufruf geantwortet hat, nicht dass die Antwort stimmt. Deshalb bewertet das Eval den Inhalt zusätzlich mit Code-Checks und Judge.

(d) Ein Lauf braucht $4+F+1$ Runden, wenn $F$ sichtbare Fehler auftreten. Aus $4+F+1 \le 6$ folgt $\boxed{F \le 1}$. Kein Fehler: $0{,}95^4 = 0{,}8145$. Genau ein Fehler: Er kann in jedem der vier Schritte liegen, also $4 \cdot 0{,}05 \cdot 0{,}95^4 = 0{,}1629$. Damit

$$P(\texttt{limit}) = 1 - 0{,}8145 - 0{,}1629 = \boxed{0{,}0226} \approx 2{,}3\,\%.$$

Etwa jeder 44. Lauf würde abgeschnitten, obwohl er sonst weitergelaufen wäre. Das Limit 6 liegt zu knapp über den mindestens nötigen 5 Runden. Der Kurs setzt deshalb 10, etwas über der typischen Schrittzahl.

> **Typischer Fehler**
>
> Die Fehlerwahrscheinlichkeiten addieren statt die Erfolgswahrscheinlichkeiten zu multiplizieren: $1 - 4 \cdot 0{,}06 = 0{,}76$ statt $0{,}94^4 = 0{,}781$. Bei acht Schritten wird der Unterschied größer (0,52 statt 0,610). Ebenfalls falsch: stille Fehler wie sichtbare behandeln und annehmen, Wiederholungen könnten sie beheben.

**Bewertung:** (a) beide Werte 1 P, Begründung zum Workflow 1 P. (b) Herleitung 1 P, $P_1$, $P_\infty$ und Laufwerte 1 P, Anteil 4,1 % mit Begründung 1 P. (c) 5,21 Runden 1 P, stille Fehler und Status `fertig` 1 P. (d) $F \le 1$ 1 P, $P(\texttt{limit})$ 1 P.

## Lösung zu Aufgabe 5 Trace prüfen, ReAct und Planung messen

(a) *Überflüssige Aufrufe:* Runde 2 wiederholt denselben Aufruf ohne neue Information. *Fehlende Prüfungen:* Status und offene Tickets wurden nie abgefragt, das Handbuch nie durchsucht. *Gedeckte Aussagen:*

| Aussage in der Antwort | Beleg im Trace | gedeckt? |
|---|---|---|
| Vibration 4,8 mm/s über Grenzwert 4,5 | Runde 1: ÜBERSCHRITTEN | ja |
| Spindeltemperatur 63 °C kritisch hoch | Runde 1: 63 °C bei max. 70, ok | nein, widerspricht der Beobachtung |
| Rundlauf zuletzt 7 µm | Runde 3: 7 µm, nahe am Grenzwert 8 | ja |
| Spindellager verschlissen | Folgerung aus Vibration und Rundlauf | teilweise: als wahrscheinliche Ursache vertretbar, nicht als Gewissheit |
| „Laut Handbuch“ stillsetzen, HTD-1100 einbauen | keine Handbuch-Suche im Trace | nein, erfunden |
| Ticket mit Priorität hoch | Vorschlag, keine Tatsache; ob schon ein Ticket offen ist, wurde nie geprüft | teilweise: als Vorschlag zulässig, aber ungeprüft |

Zusätzlich nötig: `get_machine_status(M-03)`, um offene Tickets zu sehen (in den Kursdaten gibt es für M-03 das offene Ticket S-021 mit bereits eingeplantem Lagerwechsel), und `search_knowledge_base`, um die Maßnahme bei vibrierender Spindel zu belegen. Zur Information: Im Kurs-Handbuch steht dazu „Drehzahl reduzieren, Instandhaltung informieren; Rundlauf messen, Lager nachschmieren oder tauschen“. Die Teilenummer HTD-1100 stimmt zwar, „sofort stillsetzen“ steht dort aber nicht. Auch eine zufällig richtige Angabe bleibt ein Fehler, solange sie nicht belegt ist.

(b) **Nein.** Vor Runde 2 steht `get_sensor_data(M-03)` einmal in `aufrufe`. Die Zählung ergibt $1 < 2$, der Aufruf wird ausgeführt. Erst ein dritter identischer Aufruf hätte den Abbruch ausgelöst. Die Schleifenerkennung ist ein Notausgang, sie verhindert keine einzelne Wiederholung.

(c) Der Gedanke ist ebenfalls erzeugter Text, keine garantiert wahre Erklärung. Hier begründet er nichts Neues: Dieselbe Abfrage liefert dieselben Werte, die Beobachtung aus Runde 1 war schon vollständig. Der Aufruf kostet nur eine Runde und Tokens. Ein besserer Gedanke nach dem Muster „was weiß ich, was fehlt, welches Tool als Nächstes“:

„Die Vibration liegt über dem Grenzwert, Temperatur und Hydraulikdruck sind in Ordnung. Als Ursache kommen die Spindellager in Frage. Mir fehlen offene Tickets und die letzten Arbeiten an den Lagern. Als Nächstes rufe ich `get_machine_status(M-03)` auf, danach die Historie und das Handbuch.“

(d) Nur mit Plan erfolgreich sind A4 und A7, umgekehrt keine: $\boxed{b = 0,\ c = 2}$. Exakter McNemar-Test:

$$p = \min\left(1,\ 2\sum_{i=0}^{0}\binom{2}{i}0{,}5^2\right) = 2 \cdot 0{,}25 = \boxed{0{,}5}.$$

Mehrkosten ohne Planer: $10\,000/8000 - 1 = \boxed{25\,\%}$. Mit Planer: $(10\,000 + 1700)/8000 - 1 = 0{,}4625$, also $\boxed{\text{etwa } 46\,\%}$.

Entscheidung: Ein Erfolgsunterschied ist nicht belegt ($p = 0{,}5$, die Wilson-Intervalle überlappen stark). Deutlich sind dagegen die Mehrkosten von fast der Hälfte. Auffällig ist, dass beide Gewinne Aufgaben mit vielen gleichartigen Teilschritten sind (A4, A7). Genau dort erwartet die Lerneinheit einen Nutzen. Eine gute Entscheidung: ReAct bleibt der Standard, Planung nur für Aufgaben mit vielen gleichartigen Teilschritten. Nächster Schritt: die Traces von A4 und A7 lesen (hat der Plan verhindert, dass Maschinen vergessen werden?), mehr solche Aufgaben ins Testset aufnehmen, erneut messen und die Entscheidung mit Begründung festhalten. Andere gut begründete Entscheidungen, die den fehlenden Nachweis und die Kosten berücksichtigen, geben ebenfalls volle Punkte.

**Typischer Fehler**

Aus 7/8 gegen 5/8 schließen, Planung sei „nachweislich besser“. Zwei Wechsel auf acht Aufgaben belegen nichts. Ebenso falsch: Mehrkosten nur aus der Loop-Messung ablesen und den Planer-Aufruf vergessen.

**Bewertung:** (a) drei Fragen 1 P, Tabelle 2 P (mindestens fünf Aussagen richtig bewertet; beim Ticket zählt auch „nein“, wenn begründet wird, dass der Status nie abgefragt wurde), zwei fehlende Tools mit Zweck 1 P. (b) 1 P. (c) Stellungnahme 1 P, Gedanke 1 P. (d) $b$, $c$ und $p$ 1 P, beide Mehrkosten 1 P, Entscheidung mit nächstem Schritt 1 P.

## Lösung zu Aufgabe 6 Geometrische Verteilung und Iterationslimit (Zusatz)

(a) Ableiten von $\sum_{k\ge 0} x^k = \frac{1}{1-x}$ ergibt $\sum_{k\ge 1} k\,x^{k-1} = \frac{1}{(1-x)^2}$ für $|x|<1$. Mit $x = 1-q$:

$$E(N) = \sum_{k\ge 1} k\,(1-q)^{k-1}\,q = q\cdot\frac{1}{\bigl(1-(1-q)\bigr)^2} = \frac{q}{q^2} = \boxed{\frac{1}{q}}.$$

(b) Ein Lauf mit $f$ sichtbaren Fehlern besteht aus $f$ Fehlversuchen und vier abschließenden Versuchen ohne sichtbaren Fehler. Jede konkrete Verteilung hat die Wahrscheinlichkeit $e^f(1-e)^4$. Es gibt $\binom{f+3}{3}$ Möglichkeiten, $f$ Fehler auf vier Schritte zu verteilen (Sterne und Striche: $f$ Sterne, 3 Trennstriche). Für `limit` gilt $4+F+1 > $ `max_schritte`:

Limit 7: $P(F\le 2) = 0{,}95^4\,(1+4\cdot 0{,}05+10\cdot 0{,}05^2) = 0{,}8145\cdot 1{,}225 = 0{,}9978$,
$P(\texttt{limit}) \approx \boxed{0{,}0022}$,

Limit 8: $P(F\le 3) = 0{,}8145\cdot(1{,}225+20\cdot 0{,}05^3) = 0{,}8145\cdot 1{,}2275 = 0{,}9998$,
$P(\texttt{limit}) \approx \boxed{0{,}0002}$.

Ein Limit von 8 schneidet in diesem Modell praktisch keinen gültigen Lauf mehr ab und liegt knapp drei Runden über dem Mittel von 5,21. Es begrenzt trotzdem die Kosten eines Laufs, der sich verrennt: Mit den Zahlen aus Aufgabe 3 sind es höchstens $S(8) = 52\,000$ Input-Tokens. Auch das Kurslimit von 10 ist gut begründet.